

The coding interview format

Week 3, Lecture 1 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- What the interviewer is actually evaluating (and what they are not)
- The 45-minute arc: four phases, four jobs
- The Google coding interview rubric: four signal dimensions
- Signal vs. noise: how to produce more of one and less of the other

Part 1: what is actually being tested

The interviewer's two questions

- Spolsky (2006): interviewers are looking for exactly two things
- Smart: can this person reason under uncertainty?
- Gets things done: will this person ship?
- Everything else in the room is noise unless it speaks to one of these

What is not being tested

- Memorization of specific algorithms
- Ability to produce a correct solution in the first 60 seconds
- Speed of typing
- Familiarity with a particular language's standard library

Trivia questions fail as signal because memorizers and thinkers score the same.

Part 2: the 45-minute arc

Phase 1: setup (0-5 minutes)

- You read the problem; the interviewer watches how you receive it
- Clarify constraints: input size, edge cases, expected output type
- Restate the problem in your own words before writing any code
- Signal produced here: communication, structure, comfort with ambiguity

Phase 2: exploration (5-15 minutes)

- Name a brute-force approach and state its complexity
- The interviewer is not looking for perfection; they are looking for process
- Think aloud: your reasoning is the primary evidence at this stage
- Mihailescu (2021): "When you understand what each dimension scores, you can consciously produce that signal."

Phase 3: coding (15-40 minutes)

- Start from the approach you verbally committed to
- Write readable code, not clever code: variable names matter
- Narrate as you go, especially when you hit a decision point
- If you get stuck, say what you know and what you do not know

Phase 4: review (40-45 minutes)

- Walk through your solution with a concrete example
- Name edge cases you have not handled; offer to add them
- State the final time and space complexity unprompted
- This phase distinguishes candidates who can evaluate their own work

Part 3: the interviewer rubric

Four signal dimensions (Google rubric)

- **Problem solving:** can the candidate break a problem into tractable steps?
- **Communication:** are they narrating their reasoning in real time?
- **Code quality:** is the code readable, correct, and free of obvious bugs?
- **Testing:** do they verify their solution and enumerate edge cases?

Each dimension is scored independently (Mihailescu, 2021).

Problem solving in practice

- Not: did you arrive at the optimal solution?
- Yes: did you show a path from problem to solution that a senior engineer would recognize?
- Brute force to optimized is a valid path, and the transition itself is evidence
- A candidate who jumps directly to the optimal but cannot explain the jump scores lower than one who shows the reasoning

Code quality as a signal

- Interviewers read your code as if they will maintain it
- Short, meaningful variable names outperform single letters except for trivial loop counters
- No magic numbers: name constants
- No dead code left over from debugging: clean as you go
- A correct but unreadable solution is a yellow flag at every company above entry-level

Part 4: signal vs. noise

What noise looks like

- Long silence with no verbal output (the interviewer cannot score communication)
- Coding a solution before stating an approach (removes the problem-solving signal)
- Rewriting large sections silently without explaining the change
- Asking clarifying questions after you have already started coding

None of these are fatal. All of them are unnecessary.

Producing clean signal under pressure

- Write the approach in plain English before you write any code
- Say “I’m going to think about this for 30 seconds” rather than going silent
- When stuck, name the thing you are stuck on: “I know I need $O(n)$ here but I haven’t seen how to get the lookup in constant time yet”
- Mihailescu (2021): most candidates fail not because they cannot code, but because they have no process for getting unstuck

You're looking for people who are Smart, and Get things done. It is much, much better to reject a good candidate than to accept a bad candidate.

Take-aways

1. The interview tests reasoning and communication, not memorization. A clear process matters more than a fast correct answer.
2. The 45-minute arc has four phases. Each phase produces different signal; you can prepare for each one separately.
3. The Google rubric scores four dimensions independently. A weak score on communication can cancel a strong score on problem solving.
4. Noise is self-inflicted. Long silences, premature coding, and unexplained rewrites all suppress your signal without helping your score.

What's next

- **Reading:** Week 3 reading covers all three patterns with worked code examples (read before section)
- **Lecture 2 (this week):** Two-pointer, hash map, and sliding window, with pattern-recognition practice
- **Section (this week):** Pattern sprint 1, three timed mediums with structured debrief
- **HW 1 due this week:** Resume and portfolio audit
- **HW 2 out this week:** 30-problem LeetCode pattern sprint